

Binary Search Pattern & Algorithm Guide

Mastering logarithmic divide-and-conquer search strategy

1. Core Intuition & Mechanism

Binary Search is an efficient algorithm for searching a target value within a sorted collection or monotonic decision space. Rather than scanning sequentially in $O(n)$ time, binary search repeatedly divides the search space in half.

- **Pointers (l , r , mid):** We maintain two pointers, l (low/left boundary) and r (high/right boundary), defining the current candidate search space $[l, r]$. In each iteration, we pick the midpoint $mid = l + (r - l) // 2$.
- **Halving Decision:** We compare `nums[mid]` against the target:
 - If `nums[mid] == target`: Target is found immediately.
 - If `nums[mid] > target`: Because the array is sorted, every element to the right of `mid` is also strictly larger than the target. We safely discard the right half by setting `r = mid - 1`.
 - If `nums[mid] < target`: Every element to the left of `mid` is strictly smaller. We discard the left half by setting `l = mid + 1`.

2. Complexity Analysis

TIME
COMPLEXITY
 $O(\log n)$

SPACE
COMPLEXITY
 $O(1)$

Why $O(\log n)$? Starting with n elements, each step eliminates half: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$. The maximum number of comparisons required to reduce the search space to 1 element is $\log_2(n)$. Iterative binary search uses $O(1)$ auxiliary space as it only requires a few integer pointers.

3. Step-by-Step Worked Example

Search for `target = 23` in array: `nums = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` ($n = 10$).

Step	Search Space $[l, r]$	l	r	mid	<code>nums[mid]</code>	Comparison / Decision
1	[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]	0	9	4	16	$16 < 23 \rightarrow$ Target is larger. Set <code>l = mid + 1</code> (5).
2	[23, 38, 56, 72, 91]	5	9	7	56	$56 > 23 \rightarrow$ Target is smaller. Set <code>r = mid - 1</code> (6).
3	[23, 38]	5	6	5	23	$23 == 23 \rightarrow$ Match found at index 5!

4. Python Implementation

```
class Solution(object):  
    def search(self, nums, target):  
        n = len(nums)  
        l = 0  
        r = n - 1  
        while l < r:  
            mid = l + (r - l) // 2  
            if nums[mid] > target:  
                r = mid - 1  
            elif nums[mid] < target:  
                l = mid + 1  
            else:  
                return mid  
        if nums[l] == target:  
            return l  
        else:  
            return -1
```

5. Line-by-Line Breakdown & Deep Dive

- `r = n - 1`: Initializes the right boundary to the last valid index of the array, establishing an inclusive search range $[l, r]$.
- `mid = l + (r - l) // 2`: Mathematically equivalent to `(l + r) // 2`. In languages with fixed-size integers (e.g., C++, Java 32-bit signed integers), adding `l + r` can overflow if both indices are large. Using `l + (r - l) // 2` guarantees safety against integer overflow.

6. Pattern Recognition & Real-World Applications

Reach for binary search whenever a problem exhibits a **monotonic property** (i.e., a condition that evaluates to `False` up to a certain point and `True` thereafter, or vice versa). This allows discarding half the space based on a single comparison, even if the input is not a literally sorted array.

1. Classic Search in Sorted Array

Search Space: Array indices `[0 ... n-1]`.

Decision Function: Compare `nums[mid]` directly with target to decide left or right.

2. Rotated Sorted Array

Search Space: Rotated array indices.

Decision Function: Identify which half (`[l...mid]` or `[mid...r]`) is strictly sorted, then check if target lies within that sorted range.

3. Lower Bound / Upper Bound

Search Space: Sorted array with duplicate elements.

Decision Function: To find the first occurrence, even when `nums[mid] == target`, keep searching left (`r = mid`) to find the boundary.

4. 2D Sorted Matrix

Search Space: Flattened range `[0 ... rows*cols - 1]`.

Decision Function: Convert 1D index to 2D via `matrix[mid // cols][mid % cols]` and compare with target.

5. Find Peak Element

Search Space: Unsorted mountain/peak array.

Decision Function: Compare `nums[mid]` with `nums[mid+1]`. If increasing, peak lies to the right; if decreasing, peak lies to the left.

6. Binary Search on Answer Space

Search Space: Range of possible answer values `[min_possible, max_possible]`.

Decision Function: Helper `feasible(mid)` returns boolean whether `mid` satisfies criteria (e.g., Koko eating bananas, shipping capacity).

7. Master Template: Binary Search on Answer

Below is the standard pattern for problems where you search over a range of numerical answers (e.g., minimum speed, capacity, or threshold) rather than array indices:

```
def binary_search_on_answer(low, high):  
    def feasible(val):  
        # Returns True if 'val' satisfies problem conditions  
        pass  
    res = -1  
    l, r = low, high  
    while l <= r:  
        mid = l + (r - l) // 2  
        if feasible(mid):  
            res = mid  
            # Record candidate answer  
            r = mid - 1  
            # Try finding a smaller valid value (minimization)  
        else:  
            l = mid + 1  
            # Value too small/invalid, increase search bound  
    return res
```

8. Summary & Key Takeaways

- Identify Monotonicity:** Look for a property where valid answers form a continuous block `[FFF...TTT]` or `[TTT...FFF]`.
- Prevent Overflow:** Always calculate mid using `l + (r - l) // 2`.
- Boundary Conditions:** Pay close attention to loop termination (`l <= r` vs `l < r`) and pointer updates (`mid - 1` vs `mid`) to avoid infinite loops.